

orderRIPOZITORI

```
package jp.co.activekenshu.delivery.repository;
```

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
import jp.co.activekenshu.delivery.dto.OrderDto;
```

```
/**
 * 配送依頼情報をDBに登録するRepositoryクラスです。
 *
 * <p>
 * Repositoryとは、DB操作を担当するクラスです。
 * SQLはServletやServiceには書かず、Repositoryにまとめます。
 * </p>
 */
```

```
public class OrderRepository {
```

```
/**
 * delivery_seqから次の配送IDを取得します。
 *
 * <p>
 * delivery_idはCHAR(6)なので、
 * sequenceの値をLPADで6桁の文字列にしています。
 * 例：1 → 000001
 * </p>
 *
 * @param conn DB接続
 * @return 採番された配送ID
 * @throws SQLException SQL実行に失敗した場合
 */
```

```
public String getNextDeliveryId(Connection conn) throws SQLException {
```

```
    // Oracleのdual表を使って、sequenceから次の値を取得します。
```

```
    String sql = "SELECT LPAD(delivery_seq.NEXTVAL, 6, '0') AS delivery_id FROM dual";
```

```
    // try-with-resourcesを使うことで、PreparedStatementとResultSetを自動でcloseできます。
```

```
    try (PreparedStatement ps = conn.prepareStatement(sql);
```

```

ResultSet rs = ps.executeQuery() {

    // 結果が1件取得できた場合、delivery_idを返します。
    if (rs.next()) {
        return rs.getString("delivery_id");
    }

    // 通常ここには来ませんが、万が一取得できなかった場合は例外にします。
    throw new SQLException("配送IDの採番に失敗しました。");
}
}

/**
 * delivery_order テーブルに配送依頼情報を登録します。
 *
 * @param conn DB接続
 * @param deliveryId 配送ID
 * @param order 配送依頼情報
 * @throws SQLException SQL実行に失敗した場合
 */
public void insertOrder(Connection conn, String deliveryId, OrderDto order)
    throws SQLException {

    // delivery_order にINSERT するSQLです。
    // ? の部分にはPreparedStatementで値を入れます。
    String sql =
        "INSERT INTO delivery_order ("
        + "    delivery_id, "
        + "    customer_name, "
        + "    customer_address, "
        + "    delivery_name, "
        + "    delivery_address "
        + ") VALUES (?, ?, ?, ?, ?)";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {

        // 1番目の?に配送IDを入れます。
        ps.setString(1, deliveryId);

        // 2番目の?に依頼主名を入れます。
        ps.setString(2, order.getCustomerName());
    }
}

```

```

// 3番目の?に依頼主住所を入れます。
ps.setString(3, order.getCustomerAddress());

// 4番目の?に配送先名を入れます。
ps.setString(4, order.getDeliveryName());

// 5番目の?に配送先住所を入れます。
ps.setString(5, order.getDeliveryAddress());

// INSERT 文を実行します。
ps.executeUpdate();
}
}

/**
 * delivery_situation テーブルに配送状況の初期値を登録します。
 *
 * <p>
 * 配送依頼が登録された直後の状態として、
 * delivery_status には「10」を入れます。
 * 画像の SQL では jokyo_mst の '10' が「営業所」でした。
 * </p>
 *
 * @param conn DB 接続
 * @param deliveryId 配送 ID
 * @throws SQLException SQL 実行に失敗した場合
 */
public void insertSituation(Connection conn, String deliveryId)
    throws SQLException {

    // delivery_situation に INSERT する SQL です。
    String sql =
        "INSERT INTO delivery_situation ("
        + "    delivery_id, "
        + "    delivery_status "
        + ") VALUES (?, ?)";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {

        // 1番目の?に配送 ID を入れます。

```

```
ps.setString(1, deliveryId);
```

```
// 2番目の?に初期配送状況を入れます。
```

```
// '10' は jokyo_mst に登録されている「営業所」の状態です。
```

```
ps.setString(2, "10");
```

```
// INSERT 文を実行します。
```

```
ps.executeUpdate();
```

```
}
```

```
}
```

```
}
```